

递归及其回溯算法

引例、借书问题

[问题描述]

学校放暑假时，信息学辅导教师有n本书要分给参加培训的n个学生。如：A，B，C，D，E共5本书要分给参加培训的张、刘、王、李、孙5位学生，每人只能选1本。教师事先让每个人将自己喜爱的书填写在如下的表中，然后根据他们填写的表来分配书本，希望设计一个程序帮助教师求出可能的分配方案，使每个学生都满意。

	A	B	C	D	E
张			Y	Y	
王	Y	Y			
刘		Y	Y		
孙				Y	
李		Y			Y

输入格式：第一行一个数 n （学生的个数，书的数量）
以下共 n 行，每行 n 个0或1（由空格隔开），第 i 行数据表示第 i 个同学对所有书的喜爱情况。0表示不喜欢该书，1表示喜欢该书。

输出格式：依次输出每个学生分到的书号。

样例：

输入：book.in

输出：book.out

5

3 1 2 4 5

0 1 1 1 0

1 1 0 1 1

0 1 1 0 1

0 0 0 1 0

0 1 0 0 1

第一行：输出方案总数

以下每行输出一个可行方案、每个数字减用空格隔开
（按照字典序最小顺序输出）

思路转变

针对这个问题，发现和我们之前学习的问题都不一样，那不一样的地方在哪里呢？

这个问题不再是一句话题意了？换句话说之前讲的知识主要集中在C语言语法本身上，而题目的思维难度都非常低，基本上都是一句话题意、意思都懂关键是看用代码实现，然后我们要搞好竞赛那远远不够的。我们要学习的是用C语言解决具体实际问题，所以从现在开始我们将从语法上转变到思维上。特别是题目的思考上。所以那些如果现在为止还对前面语法不熟悉的同学，那你自己必须想办法了，必须要自己去补，否则你就只能离开信息学竞赛班了。

借书问题分析

那么针对这个问题我们该如何来分析呢？

针对这个问题，我们发现是 n 个人借 n 本书的问题，因此我们可以从每个学生出发，也可以从每本书出发。

那我们从每个学生出发思考，针对每个学生：只能借一本书，而且一本书也只能借给一个人。然而可以借的书呢有 n 本，但是必须是这个学生喜欢的。所以要想每个人都要借到满意的书，所以还需要从整体上去考虑，也就是说只有每个人都借到自己满意的书才是一个方案。

5

0 1 1 1 0

1 1 1 0 0

0 1 1 1 0

0 1 0 1 1

0 1 0 0 1

针对样例输入我们分析下第1个学生只可以借第3和第4本书，第2个学生可以借第1和第二本书，那到底第1个人和第2个人借那本书才能满足我们的要求呢？这个还需要考虑后面学生喜欢书的情况而定。

借书问题总结分析

所以总体来分析：针对第 i 个学生借书，我们要考虑以下3点

- 1、第 i 个学生可以借书的范围。
- 2、 i 是否喜欢这本书。
- 3、 i 喜欢的这本书是否还可以借（也就是前面 $i-1$ 个人还没有把这本书借走。）
- 4、考虑是否 n 个人都借到了自己满意的本书，如果是这样子，才表示得到一个借书方案

那么这个问题用什么算法思想来实现呢？

我们分析发现这里针对每个人思考的方式都是一样的，所以可以用我们学习的递归来实现。

针对第i个人借书代码描述

//第i个人可以借的书b[i]，说明前i-1个人已经借了书

```
void try(int i)
```

```
{
```

```
    if (i==n+1) {输出结果; } //前n个人都借了书
```

```
    for (int j=1;j<=n;j++) //搜索人i能够可以借的书的范围
```

```
        if (第i个人可以借第j 本书)
```

```
        {
```

```
            记录下b[i]=j;
```

```
            book[j]=false;//标记第j本数已被借:
```

```
            try(i+1);
```

```
            book[j]=true; //删除书的被借标志，别人可以借
```

```
        }
```

```
    }
```



针对第i个人借第j本书：如果第i个人借第j本书这样子一直做下去，但是不能找到1种满足题目要求的方案，则第i个人不能借第j本书，所以要把book[j]=true,第i个人尝试借下一本书。

具体实现代码分析:

```
int a[10+10][10+10]; //a[i][j]=1: 第i个人喜欢第j本书, 0表示不喜欢}
int b[10+10]; //记录分配方案:b[i]是第i个人借第b[i]本书
bool book[10+10];
/*book[[i]:能否可以借第i本书,是否他人已借此书, 初始:true,一旦有人借了:
就改为:false*/
int n; //n本书
```


void init() //初始化函数，实现数据读入，数组初始化。

```
{
    scanf("%d",&n);
    for(int i=1;i<=n;i++)
    {
        for(int j=1;j<=n;j++)
        {
            scanf("%d",&a[i][j]);
        }
    }
    memset(book,true,sizeof(book));
}
```

//打印数据的函数

```
void print()
{
    for(int i=1;i<n;i++)
    {
        printf("%d ",b[i]);
    }
    printf("%d\n",b[n]);
}
```

//第i个人可以借的书函数

```
void dfs(int x){
    if(x==n+1){
        print();
    }
    for(int i=1;i<=n;i++)
    {
        if(a[x][i]==1 && book[i])
        {
            book[i]=false;
            b[x]=i;
            dfs(x+1);
            book[i]=true;
        }
    }
}
```

程序主函数

```
int main()  
{  
    init();  
    dfs(1);  
    return 0;  
}
```

解决这类问题的方法：回溯法

回溯法

回溯法（探索与回溯法）是一种选优搜索法，又称为试探法，按选优条件向前搜索，以达到目标。但当探索到某一步时，发现原先选择并不优或达不到目标，就退回一步重新选择，这种走不通就退回再走的技术为回溯法，而满足回溯条件的某个状态的点称为“回溯点”。

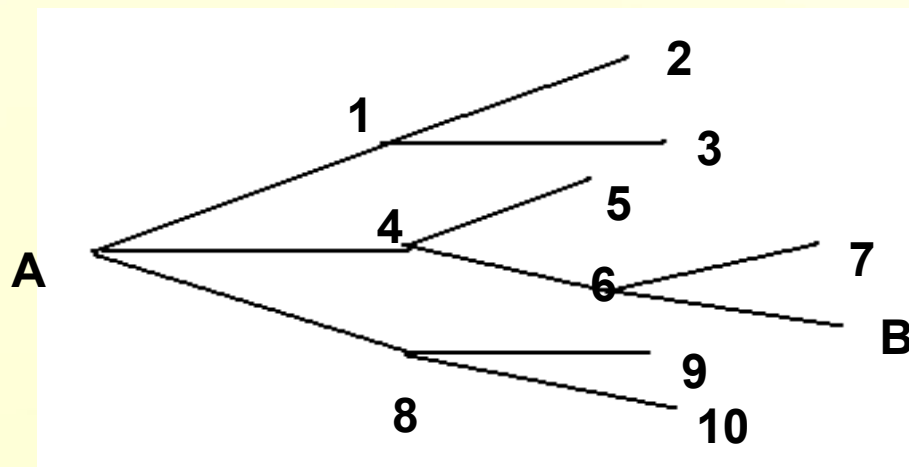
基本思想

在包含问题的所有解的解空间树中，按照深度优先搜索的策略，从根结点出发深度探索解空间树。当探索到某一结点时，要先判断该结点是否包含问题的解，如果包含，就从该结点出发继续探索下去，如果该结点不包含问题的解，则逐层向其祖先结点回溯。（其实回溯法就是对隐式图的深度优先搜索算法）。若用回溯法求问题的所有解时，要回溯到根，且根结点的所有可行的子树都要已被搜索遍才结束。而若使用回溯法求任一个解时，只要搜索到问题的一个解就可以结束

从问题的某一种可能出发, 搜索从这种情况出发所能达到的所有可能, 当这一条路走到“尽头”而没达到目的地的时候, 再倒回上一个出发点, 从另一个可能出发, 继续搜索. 这种不断“倒回一步”寻找解的方法, 称作“回溯法”.

回溯即是较简单、较常用的搜索策略, 实质就是一种搜索策略.

如: 找一条从A到B的路线



从A到B的路线:A---4---6---B

递归算法框架:

```
void dfs(int i);{搜索第i个分量 $X_i$ }
```

```
{
```

```
    if (i==n +1) print; {得到一种解}
```

```
    for  $X_i \in S_i$  且使得  $(X_1, X_2, \dots, X_{i-1}, X_i)$  //枚举 $x_i$ 分量的范围
```

```
{
```

```
    if(不冲突)
```

```
{
```

```
        记录满足条件的 $X_i$ ;
```

```
        添加相应的标志;
```

```
        try(i+1)
```

```
        删除标志;恢复之前的状态,根据具体情况选择
```

```
    }
```

```
}
```

```
}
```

例题：组合数学

在1到n的n个数中选出m个数的方案数和具体方案，
(输出方案按照字典序从小到大的顺序输出)

这个题的思考思路和选书是一模一样的，唯一不一样的地方是我们从N个数中选出M个

```
void dfs(int k,int x)//递归求解方案函数， dfs(k,x)表示第k个位置从数x开始选数
{
    if(k==m+1)//k==m表示。已经找到了m个元素，因此是一个方案，所以必须打印出来
    {
        print();//调用打印函数
    }
    else
    {
        for(int i=x;i<=n;i++)
        /*理解i=x,为什么？ 为了避免重复，所以，我们只枚举从x开始及其后面的数*/
        {
            ac[k]=i;
            dfs(k+1,i+1);
        }
    }
}
```



```
#include<stdio>
#include<algorithm>
#include<cstring>
using namespace std;
int n,m,total=0;
int ac[10];//记录组合方案的数字
void init();//文件操作函数
{
    freopen("zuhe.in","r",stdin);
    freopen("zuhe.out","w",stdout);
    scanf("%d%d",&n,&m);
}
```

```
void print();//打印组合方案函数
{
    for(int j=1;j<m;j++)
    {
        printf("%d ",ac[j]);
    }
    printf("%d\n",ac[m]);
    total++;//记录所有方案总数
}
```

```
int main()
{
    init();
    //初始化，表示一开始每个数都是可以使用的
    dfs(1,1);//dfs(i,k)表示第i个位置从数k开始选数
    printf("%d\n",total);//输出方案总数
    //while(1);
    return 0;
}
```

```

void dfs(int x,int k)//递归求解方案函数，dfs(x,k)表示从第x个数开始已找到K个数
{
    if(k==m)//k==m表示。已经找到了m个元素，因此是一个方案，所以必须打印出来
    {
        print();//调用打印函数
    }
    else
    {
        for(int i=x;i<=n;i++)
        /*理解i=x,为什么？为了避免重复，所以，我们只枚举x后面的数*/
        {
            if(f[i])//判断当前i这个数是否可以使用，如果可以使用，就放到我们的方案中
            {
                ac[k]=i;//放入到方案中
                f[i]=false;//标记为不可以使用
                dfs(i+1,k+1);//继续往下找下一个数
                f[i]=true;
                /*回溯，举个例子，我们枚举1开头的数，那么3使用过就不能再使用了，
                但是如果是2开头呢？3就还可以使用，所以就把3表示回来可以使用*/
            }
        }
    }
}

```

一 N皇后问题

[问题描述]

在 $n \times n$ 的国际象棋棋盘上，放置 n 个皇后，使任何一个皇后都不能吃掉另一个，要使任何一个皇后都不能吃掉另一个，需满足的条件是：同一行、同一列、同一对角线上只能有一个皇后。求放置方法。

如： $n=4$ 时，有以下2种放置方法。

	*		
			*
*			
		*	

		*	
*			
			*
	*		

输出：

2 4 1 3

3 1 4 2

方法一：分析：

1、问题解的形式：

x:array [1..n] of integer;

{x[i]:第i个皇后放在第i行，第x[i]列，保证所有皇后不同行}

问题的解变成求 (x[1], x[2], ... X[n]) ;x[i] ∈{1,2,3,4}

4皇后问题的解： (2, 4, 1, 3) , (3, 1, 4, 2)

	*		
			*
*			
		*	

		*	
*			
			*
	*		

2、放置第 $k(1 \leq k \leq n)$ 个皇后的递归算法:

```
void dfs(int k){  
    //搜索第k个皇后所在的列 $x[k]=?$  ,前 $k-1$ 个已放好,即已求得 $x[1] \dots x[k-1]$  }  
    if  $k=n+1$  then print (输出放置方案: 数组 $x$ ) ;  
    for(int j=1;j<=n;j++) {搜索第k个皇后所在的列j}  
        if (第k个皇后能够放置在第j列)  
        {  
            放置第k个皇后在第j列( $x[k]=j$ );  
            dfs(k+1);  
        }  
    }  
}
```

3、怎样判断：第k个皇后能否放置在第j列：

```
bool place(int k,int j)
//判断第k个皇后能否放在第j列}
{
    for(int i=1;i<=k-1;i++)
        if (x[i]==j) || (abs(x[i]-j)==abs(i-k))
        {
            return false;
        }
    return true;
}
```

方法二：

方法一的缺点：每次调用函数**place** (**i,j**) 判断第**i**个皇后能否放在**j**列时：
有一个**for**循环判断，显然浪费时间。

```
bool place(int k,int j)  
//判断第k个皇后能否放在第j列}  
{  
    for(int i=1;i<=k-1;i++)  
        if (x[i]==j) || (abs(x[i]-j)==abs(i-k))  
        {  
            return false;  
        }  
    return true;  
}
```

```
var
  x:array[1..n] of integer;
  a:array[1..n] of boolean;
    {列控制标志: true: 可以放, false: 不能放}
  b:array[2..2*n] of boolean;
    {左上右下方斜线控制标志, true: 可以放, false: 不能放}
  c:array[1-n..n-1] of boolean;
    {左下右上方斜线控制标志, true: 可以放, false: 不能放}
```

初始时:

```
  fillchar(x,sizeof(x),0);
  fillchar(a,sizeof(a),true);
  fillchar(b,sizeof(b),true);
  fillchar(c,sizeof(c),true);
```


递归算法:

```
procedure try(i:integer);
```

```
  var j:integer;
```

```
  begin
```

```
    if i=n+1 then print else
```

```
    for j:=1 to n do
```

```
      if a[j] and b[i+j] and c[i-j]
```

```
      then begin
```

```
        x[i]:=j;
```

```
        a[j]:=false;    {列控制标志}
```

```
        b[i+j]:=false; {左上右下方斜线控制标志}
```

```
        c[i-j]:=false; {左下右上方斜线控制标志}
```

```
        try(i+1);
```

```
        {如果不能递归进行, 既a[j] and b[i+j] and c[i-j]=false:
```

```
        无法放置i+1个皇后, 说明当前皇后i放置不正确, 要回溯, 消
```

```
        除标志}
```

```
        a[j]:=true;
```

```
        b[i+j]:=true;
```

```
        c[i-j]:=true
```

```
      end;
```

```
end;
```

2、无重复元素的全排列

输入n (≤ 10) 个不同的小些字母，输出n个字符的全部排列。

样例：

输入：

abc

输出：

1:abc

2:acb

3:bac

4:bca

5:cab

6:cba

```
var  
  s:string;  
  a:array[1..10] of char;{记录生成的排列序列}  
  can:array[1..10] of 0..1;{是否已在排列序列中}  
  n,i,count:integer;  
procedure print; {输出一种排列}  
  var i:integer;  
  begin  
    inc(count);  
    write(count,':');  
    for i:=1 to n do write(a[i]);  
    writeln;  
  end;
```

```
procedure try(i:integer);{开始搜第i个字符a[i]}  
var j:integer;  
begin  
  if i=n+1 then print;{得到一种排列输出数组a}  
  for j:=1 to n do    {找第i个字符}  
    if can[j]=0 then  {没进序列}  
      begin  
        a[i]:=s[j]; {加入到排序序列}  
        can[j]:=1; {标记已用了}  
        try(i+1);  {找第i+1个字符}  
        can[j]:=0; {恢复标记, 以便继续使用}  
      end;  
  end;
```

```
Begin{主程序}  
  readln(s);  
  n:=length(s);  
  fillchar(can,sizeof(can),0);  
  count:=0;  
  try(1);{开始生成第一个字符}  
end.
```

3、有重复元素的全排列

输入n (≤ 10) 个小些字母（可能重复），输出n个字符的全部排列。

样例：

输入：

abaab

输出：

1:aaabb

2:aabab

3:aabba

4:abaab

5:ababa

6:abbaa

7:baaab

8:baaba

9:babaa

10:bbaaa

分析:

对于某个字符, 不能仅仅使用标志, 即使用过了, 只要还有就可以继续使用, 所以出现的字符按类存储, 同时记录个数。

{有重复元素的排列方法一}

var

s:string;

a:array[1..10] of char; {记录生成排列}

num:array['a' .. 'z'] of integer; {相应字符出现的次数}

n,i,count:integer;

procedure print;{读入字符}

var i:integer;

begin

inc(count);

write(count,':');

for i:=1 to n do write(a[i]);

writeln;

end;

```
procedure try(i:integer);{搜索生成第i个字符}
var j:char;
begin
  if i=n+1 then print;
  for j:='a' to 'z' do
    if num[j]>0 then{只要还有没拿的}
    begin
      a[i]:=j;      {记下当前字符}
      dec(num[j]); {当前字符数量减少一个}
      try(i+1);     {找下一个}
      inc(num[j]); {恢复原来数量}
    end;
  end;
end;
```



```
Begin{主程序}  
  readln(s);  
  n:=length(s);  
  for i:=1 to n do inc(num[s[i]]);  
    {统计出现的字符个数}  
  count:=0;  
  try(1);{找第一个}  
end.
```

4、自然数n的分解

分解（一）

输入自然数n（ $n < 100$ ），输出所有和的形式。不能重复。

如：4=1+1+2；4=1+2+1；4=2+1+1 属于一种分解形式。

样例输入：

7

输出：

1:7=1+6

2:7=1+1+5

3:7=1+1+1+4

4:7=1+1+1+1+3

5:7=1+1+1+1+1+2

6:7=1+1+1+1+1+1+1

7:7=1+1+1+2+2

8:7=1+1+2+3

9:7=1+2+4

10:7=1+2+2+2

11:7=1+3+3

12:7=2+5

13:7=2+2+3

14:7=3+4

```

var n,k,i,count:integer;
  a:array[1..100] of integer;{记录分解项}
Procedure print(x);{输出分解方案}
  var i:integer;
  begin
    inc(count);
    write(count,':',n,'=');
    for i:=1 to k do write(a[i], '+' );writeln(x);{输出分解方案}
  End;
procedure try(x:integer);{要分解x， 前面已有k项}
  var i,j:integer;
  begin
    print(x);
    for j:=a[k] to x div 2 do{分解 $x=j + (x-j)$ ： 保证非递减}
    if (j>=a[k])and(x-j>=j) then
      begin
        inc(k);    a[k]:=j;
        try(x-j);
        dec(k);
      end;
    end;
  end;

```

```
begin  
  readln(n);  
  count:=0;  
  for i:=1 to n div 2 do{先分解成i+ (n-i) 两  
项和}  
    begin  
      k:=1;{分解项}  
      a[1]:=i;  
      try(n-i);  
    end;  
end.
```

分解（二）

输入自然数n和m（n, m<100），输出所有分解，分解后的每一项都不超大于m的。不能重复。如：4=1+1+2；4=1+2+1;4=2+1+1 属于一种分解形式。

如：

输入：

7

4

输出：

1:7=1+1+1+4

2:7=1+1+1+1+3

3:7=1+1+1+1+1+2

4:7=1+1+1+1+1+1+1

5:7=1+1+1+2+2

6:7=1+1+2+3

7:7=1+2+4

8:7=1+2+2+2

9:7=1+3+3

10:7=2+2+3

11:7=3+4

```

var n,m,k,i,count:integer;
  a:array[1..100] of integer;
procedure print(i,x:integer);
var j:integer;
begin
  inc(count);
  write(count,':',n,'=');
  for i:=1 to k do write(a[i],'+');
  writeln(x);
end;
procedure try(x:integer);{x是第k+1项}
var i,j:integer;
begin
  if x<=m then print(k,x);
  for j:=a[k] to x div 2 do
    if (j>=a[k])and(x-j>=j) then
      begin
        inc(k);
        a[k]:=j;
        try(x-j);
        dec(k);
      end;
  end;
end;

```

```

begin
  readln(n);
  readln(m);
  count:=0;
  for i:=1 to n div 2 do
    begin
      k:=1;
      a[1]:=i;
      try(n-i);
    end;
  end.

```

分解（三）

输入自然数 n 和 m （ $n, m < 100$ ），输出所有分解项数不超过 m 的所有形式。
不能重复。如： $4=1+1+2$ ； $4=1+2+1$ ； $4=2+1+1$ 属于一种分解形式。

如：

输入：

7 4

输出：

1:7=1+6

2:7=1+1+5

3:7=1+1+1+4

4:7=1+1+2+3

5:7=1+2+4

6:7=1+2+2+2

7:7=1+3+3

8:7=2+5

9:7=2+2+3

10:7=3+4

```

var n,m,k,i,count:integer;
  a:array[1..100] of integer;
procedure print(i,x:integer);
  var j:integer;
  begin
    inc(count);
    write(count,':',n,'=');
    for i:=1 to k do write(a[i],'+'); writeln(x);
  end;
procedure try(x:integer);{x是第k+1项}
  var i,j:integer;
  begin
    if k<=m-1 then print(k,x);
    if k>=m then exit;{剪枝优化}
    for j:=a[k] to x div 2 do
      if (j>=a[k])and(x-j>=j) then
        begin
          inc(k);
          a[k]:=j;
          try(x-j);
          dec(k);
        end;
    end;
  end;
end;

```

```

begin
  readln(n);
  readln(m);
  count:=0;
  for i:=1 to n div 2 do
    begin
      k:=1;
      a[1]:=i;
      try(n-i);
    end;
  end.

```

算法的缺点？

改进优化？

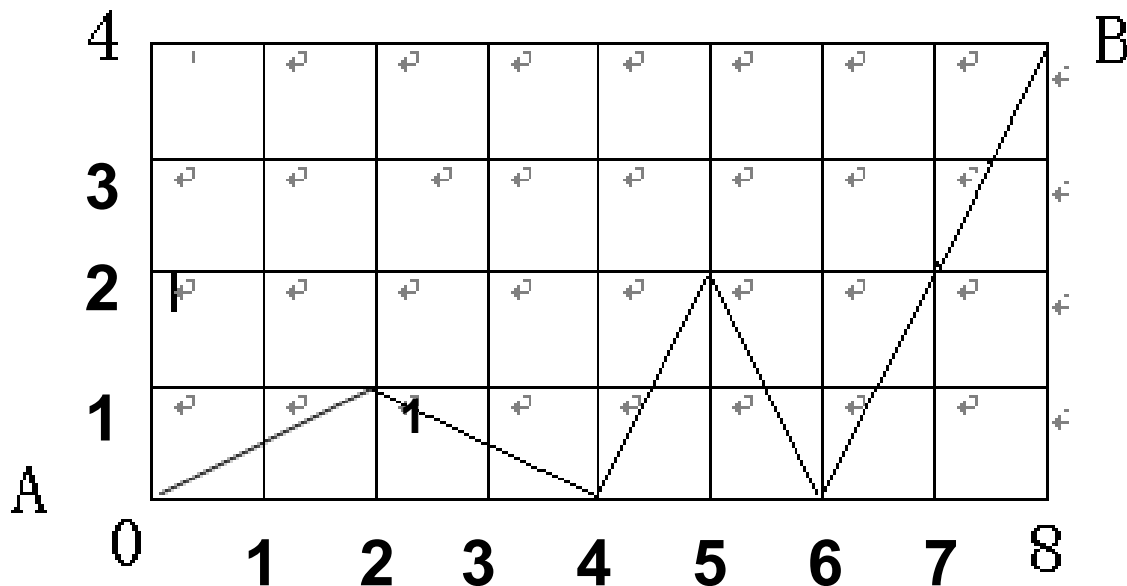
5、骑士的游历

设有下图所示的一个棋盘，在棋盘上的**A(0,0)**点有一个中国象棋马，并约定马走的规则：

1、马只向右走；

2、马走“日”字。

找出所有从**A**到**B**的路径



输入：B的坐标n，m (≤ 15)。

1

输出：所有走法。

如：

输入：8 4

输出：(0,0)(2 1)(4 0)(5 2)(6 0)(7 2)(8 4)

分析:

1、马跳的方向:

x:array[1..4,1..2]of integer=

((1,-2),(2,-1),(2,1),(1,2));

4个方向横向和纵向的增量。

2、记录马经过的位置坐标

a:array[1..16,1..2]of integer;

第i步所在的位置, 1: 横坐标

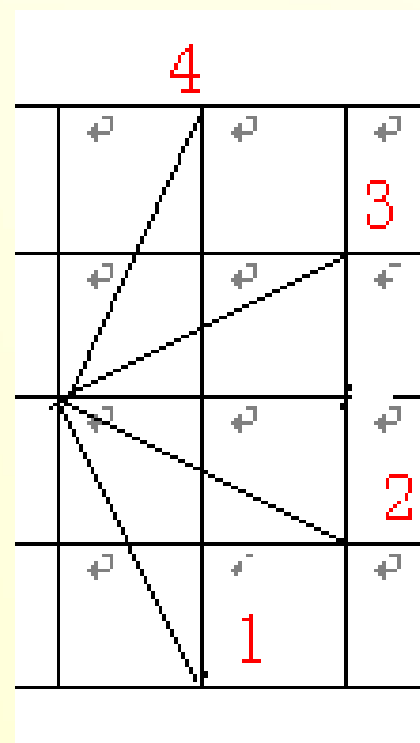
2: 纵坐标

3、马的当前位置: (**a[l,1],a[l,2]**)

下一个位置可能是:

(a[l,1]+x[j,1],a[l,2]+x[j,2]) $1 \leq j \leq 4$

4、目标: **a[l,1]=n;a[l,2]=m;**



```
const
  maxx=15;
  maxy=15;
  x:array[1..4,1..2]of integer=((1,-2),(2,-1),(2,1),(1,2)); {4个方向}
var n,m,t:integer;
  a:array[1..maxx+1,1..2]of integer;{记录走的路径坐标}
procedure print(i:integer);
var j:integer;
begin
  inc(t);
  write(t,':');
  for j:=1 to i do write('(',a[j,1],' ',a[j,2],')');
  writeln;
end;
```

```
procedure try(i:integer); {搜索到当前第i个点}  
  var j:integer;  
  begin  
    if (a[i,1]=n) and (a[i,2]=m) then print(i);  
    for j:=1 to 4 do  
      if (a[i,1]+x[j,1]>=0)and(a[i,1]+x[j,1]<=n)  
        and(a[i,2]+x[j,2]>=0)and(a[i,2]+x[j,2]<=m){判界}  
      then begin  
        a[i+1,1]:=a[i,1]+x[j,1];  
        a[i+1,2]:=a[i,2]+x[j,2];  
        try(i+1);  
      end;  
    end;  
  begin  
    assign(output,'house1.out');  
    rewrite(output);  
    readln(n,m);  
    t:=0;  
    a[1,1]:=0;{起始位置作为第1个点}  
    a[1,2]:=0;  
    try(1);  
    close(output);  
  end.
```

求最少步到达B点。

Best:最短路线，**a:** 临时得到的一个路线。**Min:** 最少步。

procedure try(i:integer); {搜索到当前第i个点}

var j:integer;

begin

if ((a[i,1]=n) and (a[i,2]=m))and(i<min) then

begin min:=i;best:=a;exit;end; {记下当前最短路径和最少步数}

if ((a[i,1]<>n) or (a[i,2]<>m))and(i>=min) then exit; {剪枝优化}

for j:=1 to 4 do

if (a[i,1]+x[j,1]>=0)and(a[i,1]+x[j,1]<=n)

and(a[i,2]+x[j,2]>=0)and(a[i,2]+x[j,2]<=m)

then begin

a[i+1,1]:=a[i,1]+x[j,1];

a[i+1,2]:=a[i,2]+x[j,2];

try(i+1);

end;

end;

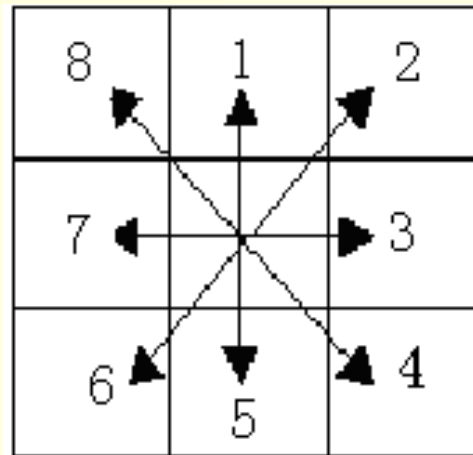
6、迷宫问题

设有一个 $N*N$ 方格的迷宫，入口和出口分别在左上角和右上角。迷宫格子中分别放有0和1，0表示可通，1表示不能，迷宫走的规则如下图所示：即从某点开始，有八个方向可走，前进方格中数字为0时表示可通过，为1时表示不可通过，要另找路径。

输入例子：（从文件中读取数据）

8

```
0 0 0 1 1 0 1 0
1 0 1 1 0 1 1 0
0 1 0 0 1 0 0 1
0 0 1 1 0 1 0 1
0 1 0 0 0 1 1 0
0 1 1 1 1 1 0 1
0 0 1 1 1 0 1 1
1 1 0 0 0 0 0 0
```



入口：（1，1）；出口：（1，8）

输出要求：找出一条 从入口（左上角）到出口（又上角）的路径(不能重复)。

(1,1)->(2,2)->(3,3)->(3,4)->(4,5)->(3,6)->(3,7)->(2,8)->(1,8)

分析:

a:array[0..maxn+1,0..maxn+1]of 0..1; {记录迷宫坐标}

b:array[0..maxn*maxn,1..2] of integer;{记录路径}

dx,dy:array[1..8]of integer; {方向位移}

8个方向的位移:

dx[1]:=0; dy[1]:=-1;

dx[2]:=1; dy[2]:=-1;

dx[3]:=1; dy[3]:=0;

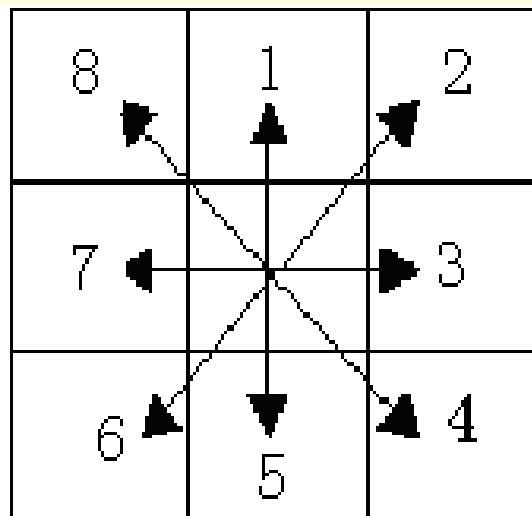
dx[4]:=1; dy[4]:=1;

dx[5]:=0; dy[5]:=1;

dx[6]:=-1; dy[6]:=1;

dx[7]:=-1; dy[7]:=0;

dx[8]:=-1; dy[8]:=-1;



```
const
  maxn=20;{递归算法}
  dx:array[1..8]of integer=(0,1,1,1,0,-1,-1,-1);
  dy:array[1..8]of integer=(-1,-1,0,1,1,1,0,-1);
var a:array[0..maxn+1,0..maxn+1]of 0..1;
    b:array[0..maxn*maxn,1..2] of integer;
    n,m,k,i,j,x,y:integer;
    sum:longint;
procedure init;
begin
  for i:=0 to maxn+1 do
    for j:=0 to maxn+1 do a[i,j]:=1;
  readln(n);
  for i:=1 to n do
    for j:=1 to n do  read(a[i,j]);
End;
```



```
procedure print(i:integer);  
  var j:integer;  
  begin  
    inc(sum);  
    write(sum,':');  
    for j:=1 to i do  
      write('(',b[j,1],', ',b[j,2],')');  
    writeln;  
  end;
```

```
procedure try(i:integer);  
  var k:integer;  
  begin  
    if (b[i,1]=1)and(b[i,2]=n) then print(i);  
    for k:=1 to 8 do  
      begin  
        if a[b[i,1]+dx[k],b[i,2]+dy[k]]=0 then  
          begin  
            b[i+1,1]:=b[i,1]+dx[k];  
            b[i+1,2]:=b[i,2]+dy[k];  
            a[b[i+1,1],b[i+1,2]]:=1;  
            try(i+1);  
            a[b[i+1,1],b[i+1,2]]:=0;  
          end;  
        end;  
      end;  
    end;  
  end;
```

```
Begin{main}  
  init;  
  sum:=0;  
  b[1,1]:=1;  
  b[1,2]:=1;  
  a[1,1]:=1;  
  try(1);  
  if sum=0 then writeln('no answer');  
end.
```

7、细胞

一矩形阵列由数字0到9组成, 数字1到9代表细胞, 细胞的定义为沿细胞数字上下左右还是细胞数字则为同一细胞, 求给定矩形阵列的细胞个数。

输入: 整数m, n (m行, n列)

矩阵

输出: 细胞的个数。

样例:

输入:

```
4 10
0234500067
1034560500
2045600671
0000000089
```

输出: 4

0 2 3 4 5 0 0 0 6 7

1 0 3 4 5 6 0 5 0 0

2 0 4 5 6 0 0 6 7 1

0 0 0 0 0 0 0 0 8 9

```
program xibao;{细胞个数}
const dx:array[1..4] of -1..1=(-1,0,1,0);
      dy:array[1..4] of -1..1=(0,1,0,-1);
var
  s:string;
  pic:array[1..50,1..80] of 0..1;
  m,n,i,j,num:integer;
  h:array[1..4000,1..2] of byte; {队列: 存细胞的坐标}
procedure init;
begin
  fillchar(pic,sizeof(pic),0);num:=0;
  fillchar(h,sizeof(h),0);
  assign(input,'b5.in');reset(input);
  readln(m,n);
  for i:=1 to m do
    begin
      readln(s);
      for j:=1 to n do
        if s[j]='0' then pic[i,j]:=0 else pic[i,j]:=1;
      end;
    end;
  close(input);
end;
```

```
procedure try(i,j:integer);  
  var k,t,w,x,y:integer;  
  begin  
    pic[i,j]:=0;  
    for k:=1 to 4 do {沿细胞的上下左右四个方向搜索细胞}  
      begin  
        x:=i+dx[k];y:=j+dy[k];  
        if (x>=1) and (x<=m) and (y>=1) and (y<=n) and (pic[x,y]=1)  
        then  
          begin  
            pic[x,y]:=0;  
            try(x,y);  
            end;{为细胞的入队}  
          end;  
        end;  
      end;
```

```
Begin {主程序}  
  init;  
  for i:=1 to m do  
    for j:=1 to n do  
      if pic[i,j]=1 then  
        begin  
          try(i,j);{在矩阵中寻找细胞}  
          inc(num);  
        end;  
    writeln(num);  
end.
```


8、组合

(一) 无重复元素的组合

输入一串小些字母（无重复字母），从中取出k个字母，输出组合情况。

样例：

输入：

abcd

3

输出：

abc

abd

acd

bcd

```
var
  s:string;
  n,m:integer;
  a:array[1..10]of char;
procedure print;
  var j:integer;
  begin
    for j:=1 to m do write(a[j]); writeln;
  end;
procedure try(i,k:integer);
  {搜索组合的第p个元素, x搜索位置}
  var j:integer;
  begin
    if i=m+1 then print;
    for j:=k to n do
      begin
        a[i]:=s[j];
        try(i+1,j+1);
      end;
    end;
end;
```

```
begin
  readln(s);
  readln(m);
  n:=length(s);
  try(1,1);
end.
```

(二)、有重复元素的组合

输入一串小些字母（有重复字母），从中取出k个字母，输出组合情况。

样例：

输入：

aabbcc

4

输出：

1:aabb

2:aabc

3:aacc

4:abbc

5:abcc

6:bbcc

9、数字排列(li3.txt)

在一个 $N*N$ 的棋盘上 ($1 \leq n \leq 10$), 填入 $1, 2, \dots, n*n$ 共 $n*n$ 个数, 使得任意两个相邻的数之和为素数

例如:

$n=2$ 时, 有:

1 2

4 3

$n=4$

1 2 11 12

16 15 8 5

13 4 9 14

6 7 10 3

分析：

逐行搜索1到 $n*n$ 之间的数放在 (i, j) 处，依次判断他与上方 $(i-1, j)$ 的数和左边 $(i, j-1)$ 的数的和是否为素数，是就放 (i, j) 处，再放 $(i, j+1)$ ；如果不是素数，则继续在1到 $n*n$ 之间搜索合适的数能放在 (i, j) 处。

```
const maxn=100;
var i,j,k,m,n,x,y,nn:integer;
    p: array[1..2*maxn*maxn]of boolean;
    {素数表: p[i]=true :i是素数,  p[i]=false :i不是素数}
    b:array[1..maxn,1..maxn]of integer;{坐标}
    used:array[1..maxn*maxn]of boolean;{检查是否该数是否用过}
```

{筛选法创建素数表}

procedure prime;

var i,j,s:integer;

begin

fillchar(p,sizeof(p),true);

p[1]:=false;

for i:=2 to $3*n \div 2$ do {依次搜索素数i并筛掉是i倍数的数}

if p[i] then

begin

j:=2*i;

while j<=2*n*n do

begin

p[j]:=false;

j:=j+i;

end;

end;

end;

判断 (x,y) 位置能否放数值k的函数

function ok(x,y,k:integer):boolean;

begin

ok:=true;

if x>1 then if not (p[b[x-1,y]+k]) then ok:=false;

if y>1 then if not (p[b[x,y-1]+k]) then ok:=false;

end;

```
procedure try(x,y,dep:integer);{递归搜索 (x,y) 处放第 dep 个数}
var i:integer;
begin
  if dep=n*n+1 then print; {如果已放了n*n个数，得出一种方法}
  for i:=1 to n*n do
    if not(used[i]) and ok(x,y,i) then
      begin
        b[x,y]:=i;
        used[i]:=true;
        if y=n then try(x+1,1,dep+1) {如果当前是最右边一列，则转到下一行首列}
        else
          try(x,y+1,dep+1); {继续放当前行的下一列}
        used[i]:=false;{释放标志}
      end;
  end;
end;
```



```
procedure print;  
  var i,j:integer;  
begin  
  for i:=1 to n do begin  
    for j:=1 to n do write(b[i,j]:4);  
    writeln;  
  end;  
  halt;  
end;
```

```
主程序:  
begin  
  readln(n);  
  prime;  
  b[1,1]:=1;  
  for i:=2 to n*n do used[i]:=false;  
  used[1]:=true;  
  try(1,2,2);  
  writeln('NO');  
end.
```

```
const maxn=100;
var a:array [1..maxn] of integer;
    n,m:integer;
procedure try(i:integer);
var j:integer;
begin
    if i>n then
        begin
            write('{');
            for j:=1 to m-1 do write(a[j],',');
            if m>0 then write(a[m]);
            writeln('}');
            exit;
        end;
    try(i+1);
    inc(m);
    a[m]:=i;
    try(i+1);
    dec(m);
end;
```

```
begin
    read(n);
    m:=0;
    try(1);
end.
```

10、因式分解

输入自然数 n ($<10^9$), 将 n 分解成一系列自然数乘积的形式:

$N=a_1*a_2*.....*a_m$, $1<a_1\leq a_2\leq.....\leq a_m<n$.

输出所有的分解方案及方案数目。

样例:

输入:

12

输出:

2*6

2*2*3

3*4

3

```
var n,k,i,count:integer;  
    a:array[1..100] of integer;  
procedure print(x:integer);  
    var i:integer;  
    begin  
        inc(count);  
        write(count,':',n,'=');  
        for i:=1 to k do  
write(a[i],'*');  
writeln(x);  
    end;
```

```
procedure try(x:integer);  
    var j:integer;  
    begin  
        print(x);  
        for j:=a[k] to trunc(sqrt(x)) do  
            if (x mod j=0)and(x div j>=j) then  
                begin  
                    inc(k);  
                    a[k]:=j;  
                    try(x div j);  
                    dec(k);  
                end;  
        end;  
    end;
```

```
begin  
  readln(n);  
  count:=0;  
  for i:=2 to trunc(sqrt(n)) do  
    if n mod i=0 then  
      begin  
        k:=1;  
        a[1]:=i;  
        try(n div i);  
      end;  
    end;  
  end.
```

1)构造因子表

procedure makebiao;

var i,p:longint;

begin

k:=0;

for i:=2 to trunc(sqrt(n)) do

if n mod i =0 then

begin

inc(k);

a[k]:=i;

end;

p:=2*k;

if a[k]*a[k]=n then dec(p);

for i:=K+1 to p do a[i]:=n div a[p-i+1];

{for i:=1 to p do write(a[i],' '); writeln(p);}

end;

注意:注意 n为完全平方数时:dec(p)

N=12: 时 2 3 4 6

N=100时: 2 4 5 10 20 25 50

2)递归求解:

procedure try(j,n1,h:longint);

**{ j:待搜索的因子表起点序号,即从a[j]到a[k]中开始搜索因数表中的因数,
n1:待分解的尾因子,**

H:目前从因子表中搜索到的因子数目 :初始状态:try(1,n,0)}

var i:longint;

begin

if h>0 then

begin

inc(t);

for i:=1 to h do write(b[i],'*');writeln(n1);

end;

for i:=j to k do

if n1 div a[i]<a[i] then exit

else if n1 mod a[i]=0

then begin

b[h+1]:=a[i];

try(i,n1 div a[i],h+1);

end;

end;

```
var  
  a,b:array[1..100000] of longint;  
  n,t,k:longint;  
主程序:  
begin  
  readln(n);  
  t:=0;  
  makebiao;  
  try(1,n,0);  
  writeln(t);  
end.
```


11、工作安排

n个人从事**n**项工作，每人只能从事一项，求最佳安排使效益最高。

设有**A, B, C, D, E**五人从事**J1, J2, J3, J4, J5**五项工作，每人只能从事一项，他们的效益如下：

当**A**从事**J5**，**B**从事**J3**，
C从事**J4**，**D**从事**J1**，**E**
从事**J2**时收益最大值：**50**

输入：**n**和矩阵

输出：最大效益和方案

	J1	J2	J3	J4	J5
A	13	11	10	4	7
B	13	10	10	8	5
C	5	9	7	7	4
D	15	12	10	11	5
E	10	11	8	8	4

输入：

5

13 11 10 4 7
13 10 10 8 5
5 9 7 7 4
15 12 10 11 5
10 11 8 8 4

输出：

50

1: 5
2: 3
3: 4
4: 1
5: 2

```
const
    maxn=5;
var
    data:array[1..maxn,1..maxn] of integer;
    n,i,j,max: integer;
    f,g: array [1..maxn] of integer;
    {f: 保存临时组合; g: 保存最佳组合}
    p: array [1..maxn] of integer;{工作是否已分配}
procedure init;
begin
    assign(input,'ex8.in');
    reset(input);
    readln(n);
    for i:=1 to n do
        for j:=1 to n do read(data[i,j]);
    close(input);
end;
```

**procedure try(k,t: integer);{搜索第k个人应从事的工作，获利共为t，
初始时： try (1, 0) }**

var

i: integer;

begin

if k=n+1 then

if t>max then

begin

max:=t;

g:=f;{保存当前的最佳方案}

exit;

end;

for i:=1 to n do

if p[i]=0 then

begin

f[k]:=i;

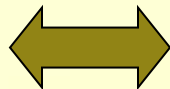
p[i]:=1;

try(k+1,t+data[k,i]);

p[i]:=0;

end;

end;



t:=t+data[k,i];

Try(k+1,t);

t:=t-data[k,i]

```
begin  
  init;  
  max:=0;  
  fillchar(p,sizeof(p),0);  
  try(1,0);  
  writeln(max);  
  for i:=1 to n do writeln(i,':',g[i]);  
end.
```

